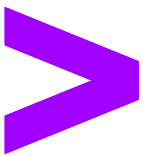


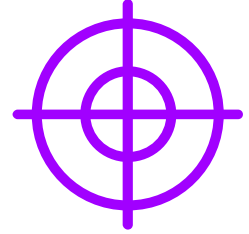
# TestNG Framework



# Module Objectives

At the end of this course, you will be able to:

- Describe the TestNG Framework and its features
- Explore the installation of the TestNG plugin for Eclipse
- Elaborate the usage of TestNG test cases along with Annotations
- Realize the functioning of Assertions using Assert class
- Describe the functional offering of Test Suite



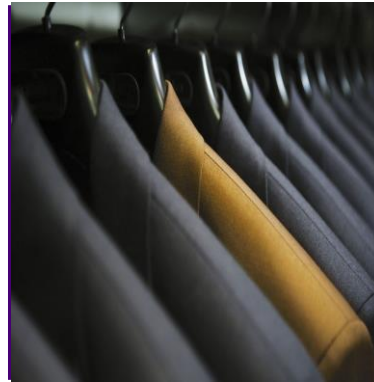
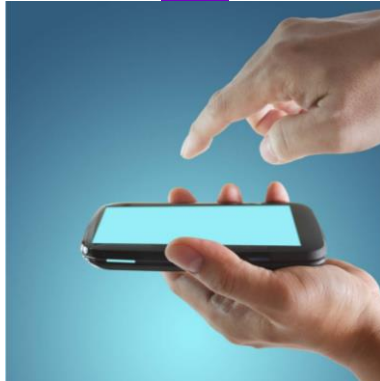
# Ground Rules

For a successful class, please:



Arrive on time

Keep your cell phones off



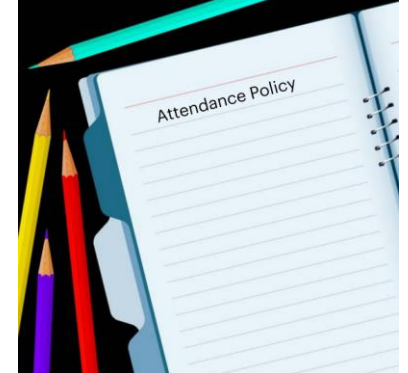
Wear business formals

Assist and show respect to your colleagues irrespective of skill and knowledge level



Do not use the web for personal reasons during training sessions

Adhere to attendance policy as directed by training coordinator



# TestNG Framework

- 01** Introduction to TestNG Framework
- 02** TestNG plugin and library for Eclipse
- 03** TestNG Test Cases with Annotations
- 04** Assertions by using Assert Class
- 05** Test Suite in TestNG
- 06** TestNG Report Generation

# Introduction to TestNG Framework

# Introduction to TestNG Framework (1)

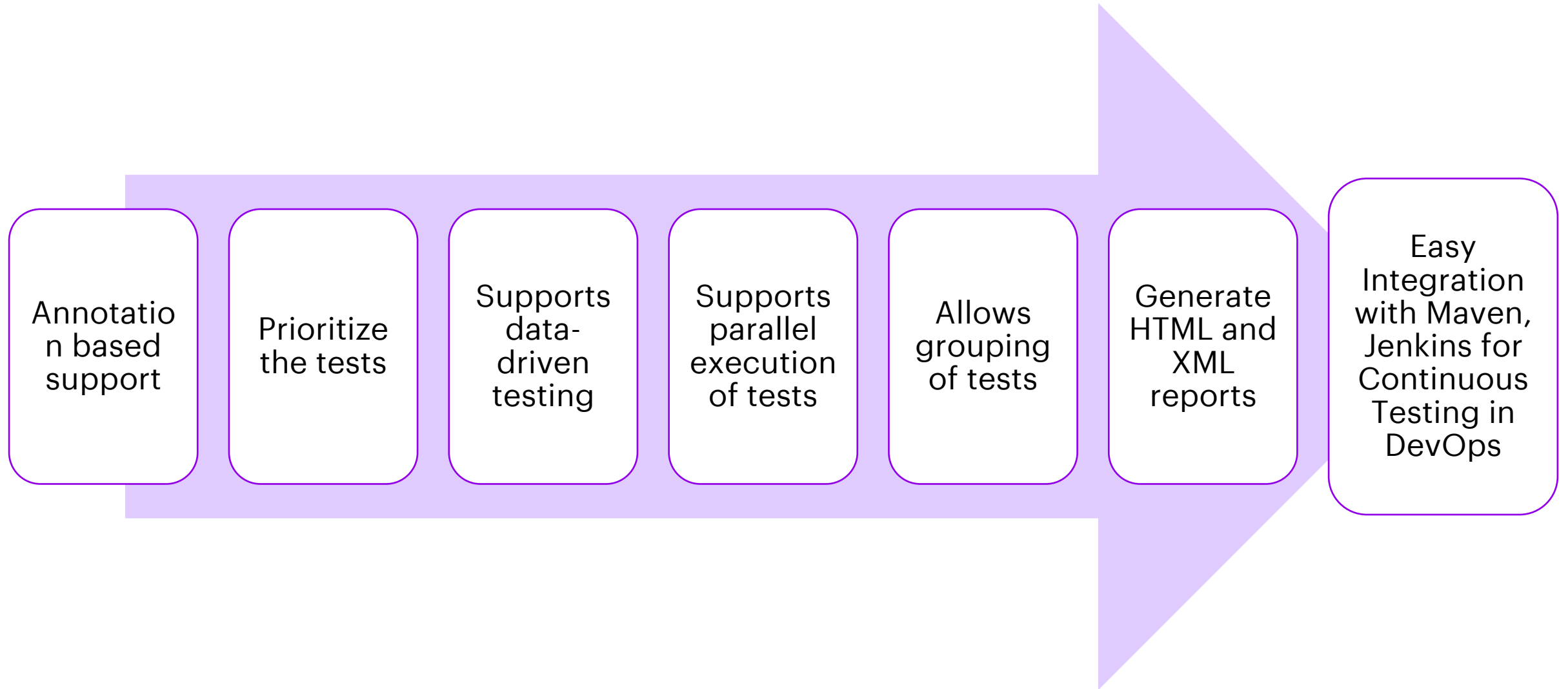
## TestNG

- **TestNG – Test Next Generation**
- A Unit testing framework, inspired from JUnit and NUnit
- It is an open-source unit testing framework, having enhanced features as compared to Junit and NUnit
- It is developed by Cedric Beust
- Useful in various test stages of STLC:
  - Unit Tests
  - Integration Tests
  - End-to-End Tests
- Pre-requisite
  - Since the framework is designed for Java, JDK must be installed in the machine



# Introduction to TestNG Framework (2)

## TestNG Features On-the-Go





# **TestNG plugin and library for Eclipse**



# TestNG plugin and library for Eclipse (1)

Pre-requisites to install TestNG Plugin

## **JDK**

<https://www.oracle.com/in/java/technologies/downloads/>

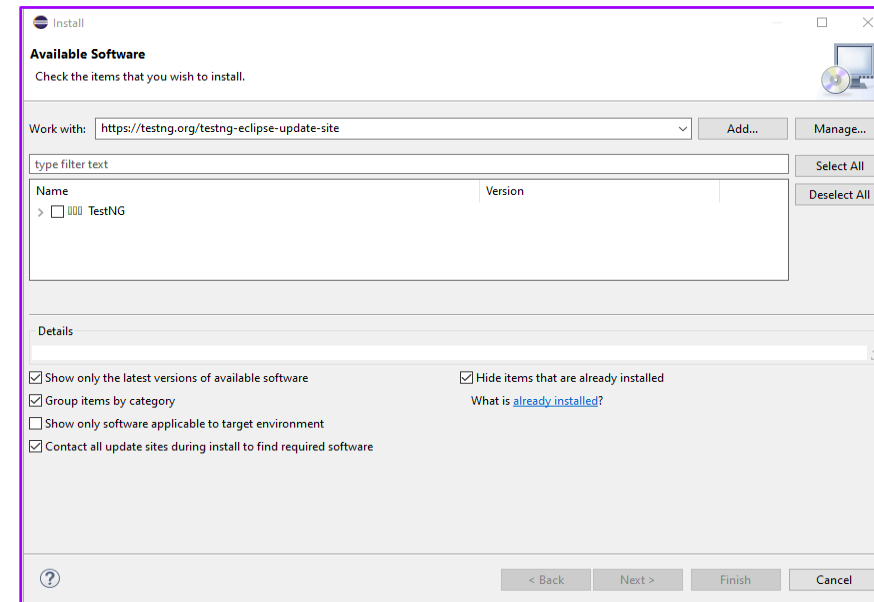
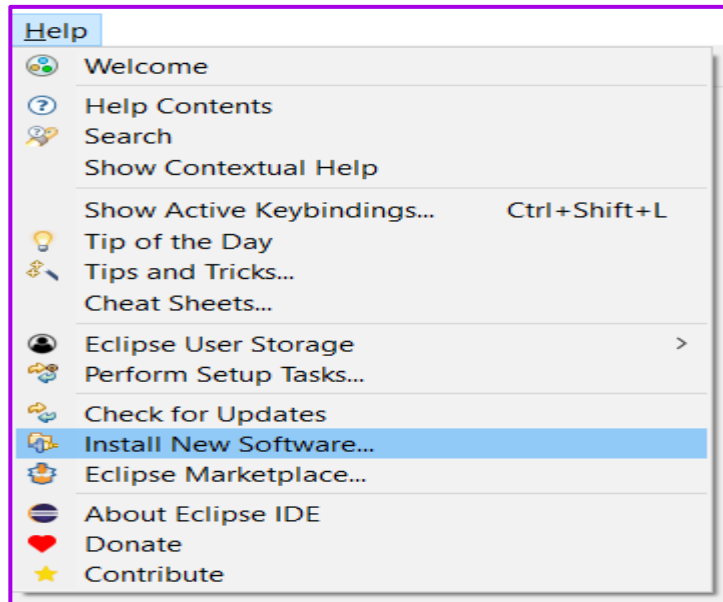
## **Eclipse**

<https://www.eclipse.org/downloads/packages/>

# TestNG plugin and library for Eclipse (2)

## Install TestNG Plugin in Eclipse

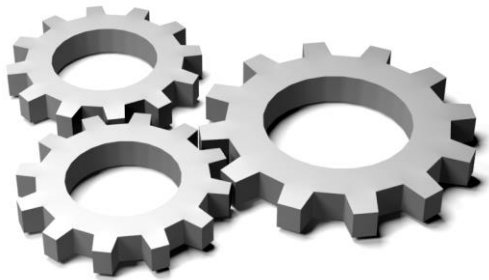
- **Step 01:** Navigate to Help Menu → Install New Software
- **Step 02:** Click on Add Button and fill below information:
  - Name → TestNG
  - Location → <https://testng.org/testng-eclipse-update-site>
- **Step 03:** Check the TestNG plugin and proceed with Installation steps



# TestNG plugin and library for Eclipse (3)

## TestNG Library

- TestNG Framework comes as a Java library for designing the unit tests
- TestNG Library contains Java predefined classes, interfaces, and annotations



**Test**

**BeforeMethod**

**AfterMethod**

**BeforeClass**

**AfterClass**

**BeforeTest**

**AfterTest**

**BeforeSuite**

**AfterSuite**

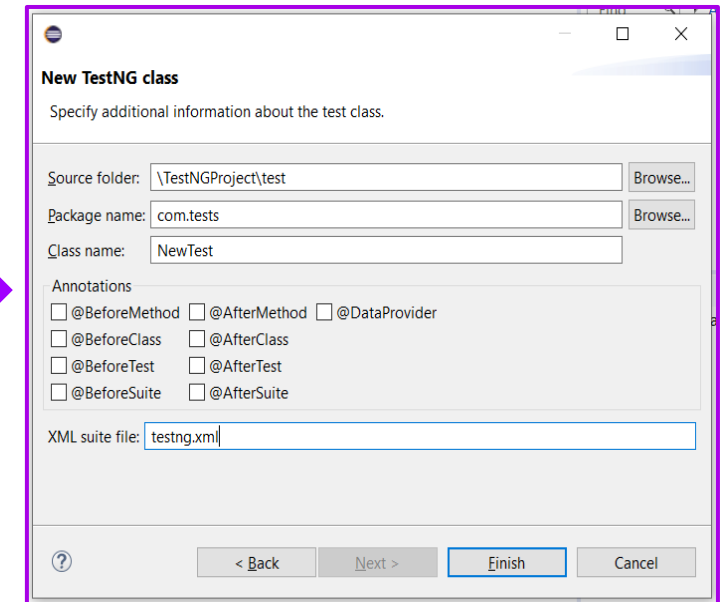
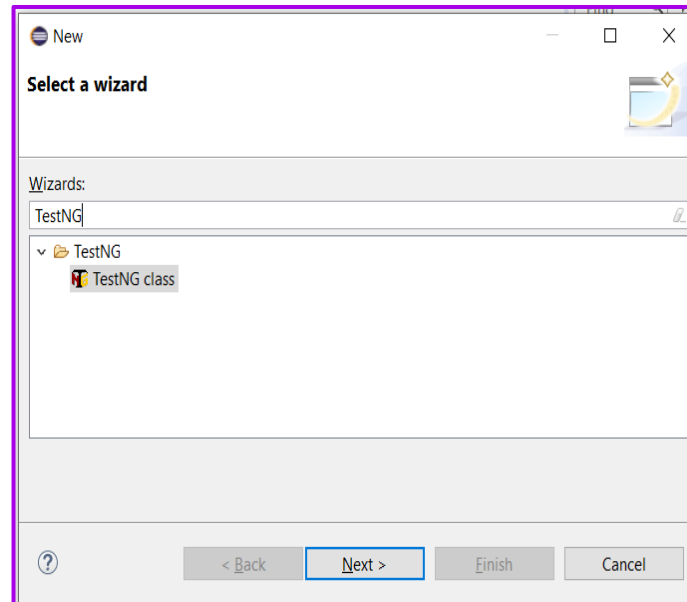
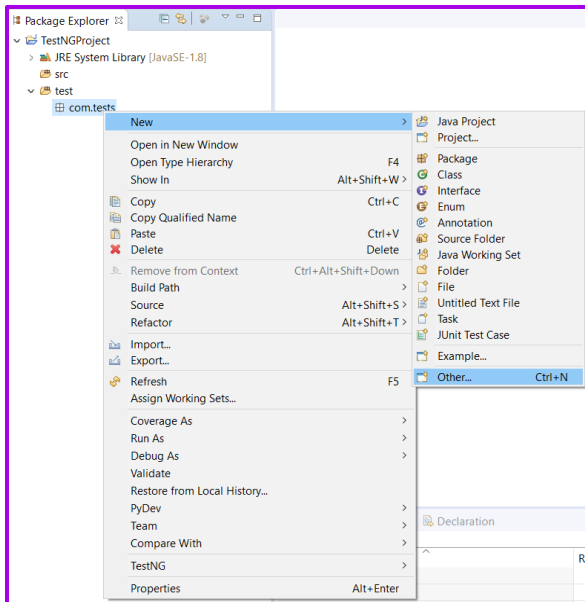
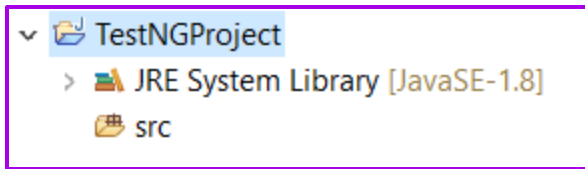
**DataProvider**

**Assert**

# TestNG plugin and library for Eclipse (4)

## Steps to create library for TestNG

- **Step 01:** Create Java Project in Eclipse (File → New → Java Project)
- **Step 02:** Create a TestNG class, provide source folder, package name, class name, and XML suite file name as shown below



# TestNG plugin and library for Eclipse (5)

## Steps to create library for TestNG

- **Step 03:** The TestNG class is created now with highlighting errors on the Test package as shown below
- **Step 04:** Resolve the error by hovering the mouse on the red lines and select Add TestNG library to add to the project as shown below

```
package com.tests;

import org.testng.annotations.Test;

public class NewTest {
    @Test
    public void f() {
    }
}
```

The import org.testng cannot be resolved

9 quick fixes available:

- Add TestNG library
- Create class 'Test' in package 'org.testng.annotations'
- Create interface 'Test' in package 'org.testng.annotations'
- Create annotation 'Test' in package 'org.testng.annotations'
- Create enum 'Test' in package 'org.testng.annotations'
- Remove unused import
- Organize imports
- Search repositories for 'org.testng.annotations'



Package Explorer

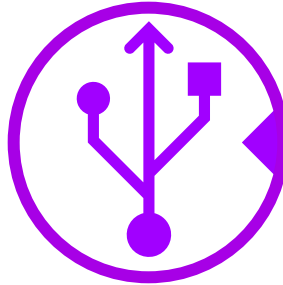
- TestNGProject
  - JRE System Library [JavaSE-1.8]
  - src
    - test
    - TestNG

testng.xml

```
1 package com.tests;
2
3 import org.testng.annotations.Test;
4
5 public class NewTest {
6     @Test
7     public void f() {
8     }
9 }
10
```

# TestNG plugin and library for Eclipse (6)

## Difference between TestNG Plugin and Library



TestNG Plugin is used for the execution of the tests. To run the test scripts as TestNG test, we need to install the TestNG plugin in eclipse



TestNG Library is used for designing the test scripts using the predefined classes, methods, and annotations

# **TestNG Test Cases with Annotations**

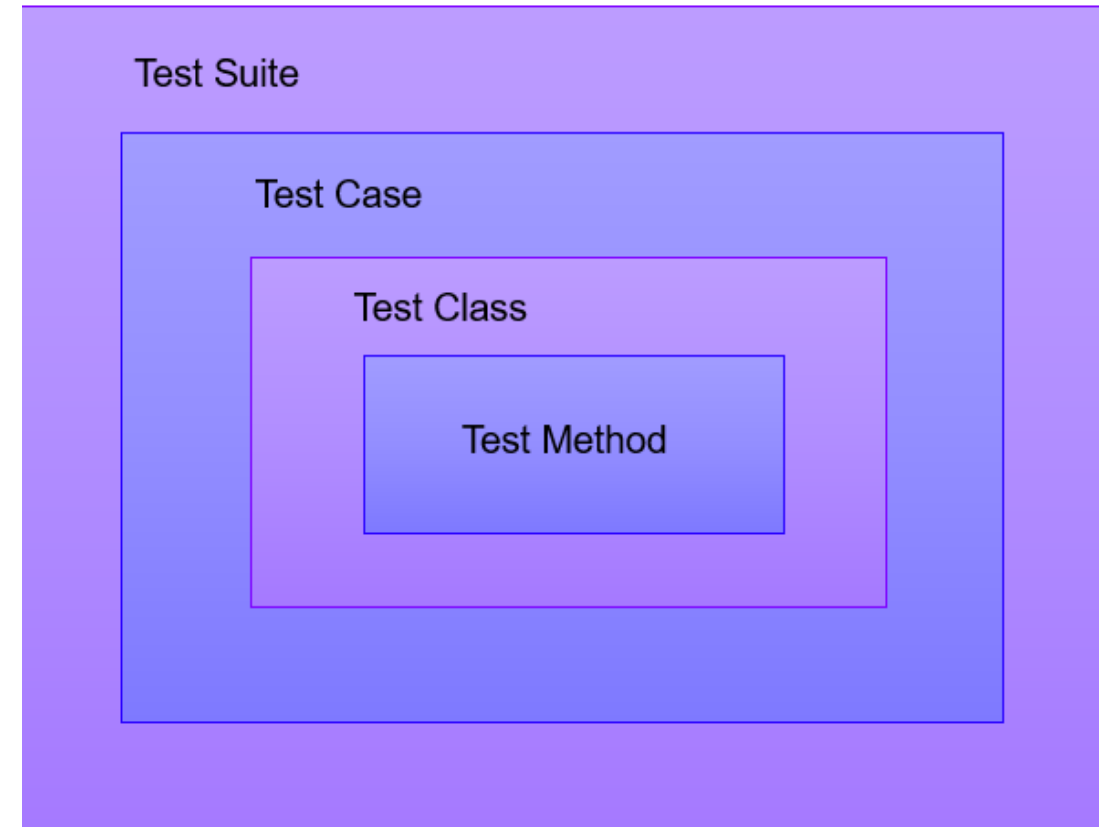


# TestNG Test Cases with Annotations (1)

## Hierarchy in TestNG

TestNG helps us to create the following

- **Test Method**
  - Contains the set of instructions to execute
- **Test Class**
  - Contains single/multiple test methods
- **Test Case**
  - Contains single/multiple test classes
- **Test Suite**
  - Contains single/multiple test cases



# TestNG Test Cases with Annotations (2)

## TestNG Annotations

### Annotations

- @Test
- @BeforeMethod, @AfterMethod
- @BeforeClass, @AfterClass
- @BeforeTest, @AfterTest
- @BeforeSuite, @AfterSuite
- @DataProvider

### Supporting Attributes

- Priority
- Enabled



# TestNG Test Cases with Annotations (3)

## TestNG Annotations - Explained

### @BeforeClass

- Marks a method to run only once, before the first test method in the current class is invoked

### @AfterClass

- Marks a method to run only once, after execution all the test methods in the current class

### @BeforeMethod

- Marks a method to run before execution of each test method

### @AfterMethod

- Marks a method to run after execution of each test method

### @Test

- Marks a class or a method for test execution

### @BeforeTest

- Marks a method to run before any test cases <test> tag in testng.xml (Test Suite)

### @AfterTest

- Marks a method to run after any test cases <test> tag in testng.xml (Test Suite)

### @BeforeSuite

- Marks a method to run only once before all the tests in testng.xml (Test Suite)

### @AfterSuite

- Marks a method to run only once after all the tests in testng.xml (Test Suite)

# TestNG Test Cases with Annotations (4)

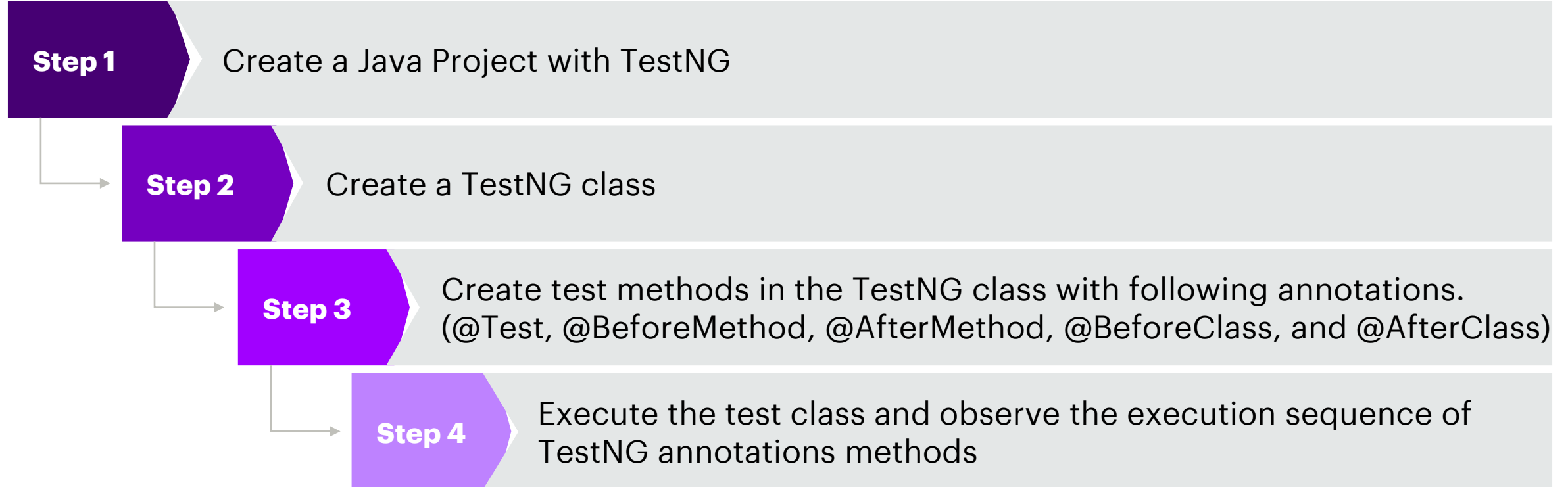
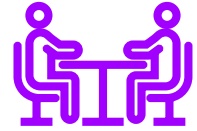
## TestNG Annotations - Usage

- **Before/After Annotations** are used for reusability of code that must execute many times but written once.
- **Before Annotations:** These are used for pre initializations like Open browser, Open File connections, etc.
- **After Annotations:** These are used for cleaning resources, post-execution of the tests like close the browser, close file connections, etc.

# TestNG Test Cases with Annotations (5)

## Demo: Execution Sequence of all TestNG Annotations

- Linda wants to check the execution sequence of all the TestNG annotations
- She decides to design a Java Project with TestNG class and annotations
- Scenario



# TestNG Test Cases with Annotations (6)

## Demo Solution: Execution Sequence of all TestNG Annotations



```
package demo;

import org.testng.annotations.AfterClass;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class TestClass01 {

    @BeforeClass
    public void beforeC() {
        System.out.println("In before class");
    }

    @AfterClass
    public void afterC() {
        System.out.println("In after class");
    }
}
```

```
@BeforeMethod
public void beforeM() {
    System.out.println("In before method");
}

@AfterMethod
public void afterM() {
    System.out.println("In after method");
}

@Test
public void method01() {
    System.out.println("In test method 01");
}

@Test
public void method02() {
    System.out.println("In test method 02");
}

@Test
public void method03() {
    System.out.println("In test method 03");
}
}
```

# TestNG Test Cases with Annotations (7)

Demo Result: Execution Sequence of all TestNG Annotations



Run As – TestNG Test

```
[RemoteTestNG] detected TestNG version 7.3.0
In before class
In before method
In test method 01
In after method
In before method
In test method 02
In after method
In before method
In test method 03
In after method
In after class
PASSED: method01
PASSED: method02
PASSED: method03

=====
      Default test
      Tests run: 3, Failures: 0, Skips: 0
=====

=====
Default suite
Total tests run: 3, Passes: 3, Failures: 0, Skips: 0
=====
```

# TestNG Test Cases with Annotations (8)

## Priority

- The Test methods in TestNG class execute in alphabetical order by default
- To customize the order of execution, pass priority in @Test as the argument with a numeric value
- The least number will have the highest priority to execute
- By default, the priority numeric value is 0 (zero)

```
@Test(priority = 0)
public void testYahoo() {
    System.out.println("In testYahoo method");
}

@Test(priority = 1)
public void testGoogle() {
    System.out.println("In testGoogle method");
}

@Test(priority = 2)
public void testMSN() {
    System.out.println("In testMSN method");
}
```

```
[RemoteTestNG] detected TestNG version 7.3.0
In testYahoo method
In testGoogle method
In testMSN method
PASSED: testYahoo
PASSED: testGoogle
PASSED: testMSN
```



# TestNG Test Cases with Annotations (9)

## Disabling a Test Method

- By default, all the test methods in a class will execute
- We can force the TestNG to ignore/disable the execution of a particular test method
- This can be implemented by passing an argument “enabled=false” in @Test

```
@Test(priority = 0)
public void testYahoo() {
    System.out.println("In testYahoo method");
}

@Test(priority = 1)
public void testGoogle() {
    System.out.println("In testGoogle method");
}

@Test(priority = 2, enabled=false)
public void testMSN() {
    System.out.println("In testMSN method");
}
```

```
[RemoteTestNG] detected TestNG version 7.3.0
In testYahoo method
In testGoogle method
PASSED: testYahoo
PASSED: testGoogle

=====
      Default test
      Tests run: 2, Failures: 0, Skips: 0
=====
```

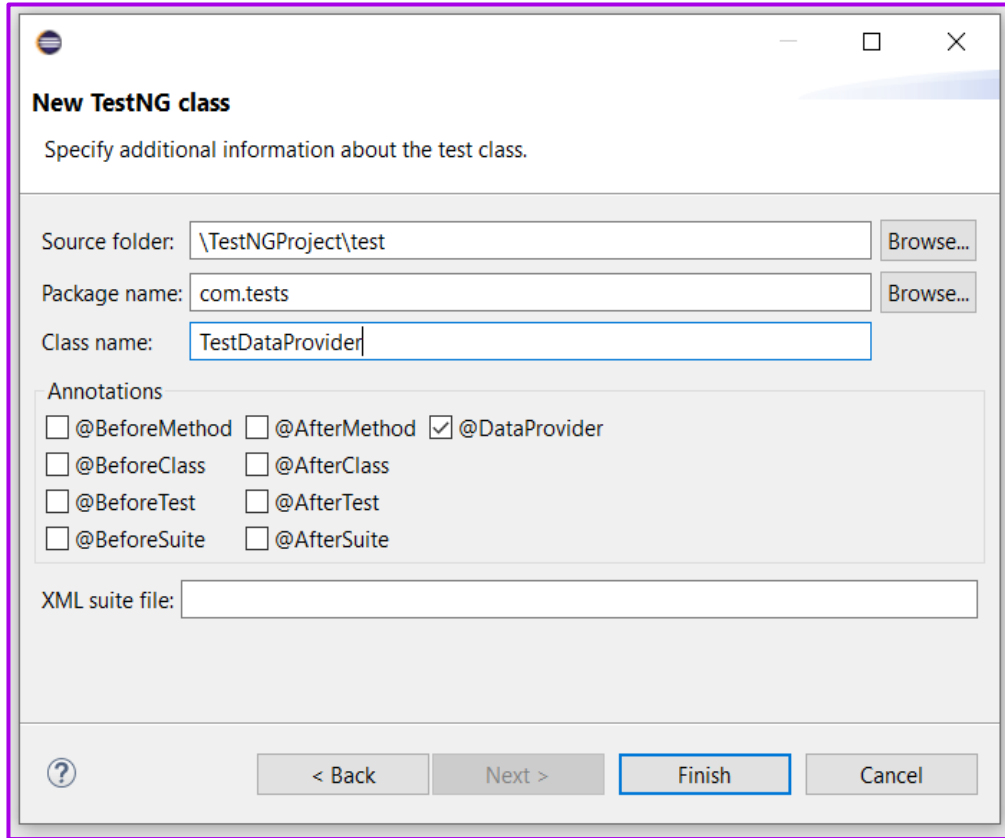
# TestNG Test Cases with Annotations (10)

## TestNG Annotation - DataProvider

- Data Provider hook is used to pass data from a source to a particular test method to replace the constant values with variables
- It internally works as a loop
- The number of iterations depends on the number of rows generated in the DataProvider method
- **@DataProvider**
  - Marks a method to provide data to the test method
  - The data provider method must return an `Object[ ][ ]`, where data is interpreted in the form rows and columns
  - The test method that wants to use the data from DataProvider, must have “DataProvider” name as an attribute for the @Test

# TestNG Test Cases with Annotations (11)

## Example: Use of @DataProvider



**New TestNG class**  
Specify additional information about the test class.

Source folder: \TestNGProject\test Browse...

Package name: com.tests Browse...

Class name: TestDataProvider

Annotations

☐ @BeforeMethod ☐ @AfterMethod ☒ @DataProvider

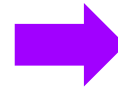
☐ @BeforeClass ☐ @AfterClass

☐ @BeforeTest ☐ @AfterTest

☐ @BeforeSuite ☐ @AfterSuite

XML suite file:

? < Back Next > Finish Cancel



```
public class TestDataProvider {  
    @Test(dataProvider = "dp")  
    public void f(Integer n, String s) {  
    }  
  
    @DataProvider  
    public Object[][] dp() {  
        return new Object[][] {  
            new Object[] { 1, "a" },  
            new Object[] { 2, "b" },  
        };  
    }  
}
```

# Assertions by using Assert Class

# Assertions by using Assert Class (1)

## Assertions

- Assertions are used to verify a particular condition
- Assertions decide the status of a test method (Pass or Fail)
- TestNG throws AssertionError, in case the assert condition returns a false. This AssertionError fails the test
- TestNG provides 2 types of assertions:
  - Hard Assertions from Assert Class
  - Soft Assertions from SoftAssert Class

# Assertions by using Assert Class (2)

## Assertion Method

- **assertTrue()** - Performs a comparison of 2 objects. The assertion passes if the comparison result is

```
@Test
public void testMethod1() {
    System.out.println("testMethod1 started");
    Assert.assertTrue(20>18, "Err-18 is less than 20");
    System.out.println("testMethod1 completed");
}

@Test
public void testMethod2() {
    System.out.println("testMethod2 started");
    Assert.assertTrue("Hello".equals("Hi"), "Err-Both strings are differ");
    System.out.println("testMethod2 completed");
}
```

```
[RemoteTestNG] detected TestNG version 6.14.2
testMethod1 started
testMethod1 completed
testMethod2 started
PASSED: testMethod1
FAILED: testMethod2
java.lang.AssertionError: Err-Both strinf are differ expected [true] but found [false]
```

# Assertions by using Assert Class (3)

## Assertion Method

- **assertFalse()** - Performs a comparison of 2 objects. The assertion passes if the comparison result is false.

```
@Test
public void testMethod1() {
    System.out.println("testMethod1 started");
    Assert.assertFalse(20>18, "Err-18 is less than 20");
    System.out.println("testMethod1 completed");
}

@Test
public void testMethod2() {
    System.out.println("testMethod2 started");
    Assert.assertFalse("Hello".equals("Hi"), "Err-Both strings are differ");
    System.out.println("testMethod2 completed");
}
```

```
[RemoteTestNG] detected TestNG version 6.14.2
testMethod1 started
testMethod2 started
testMethod2 completed
PASSED: testMethod2
FAILED: testMethod1
java.lang.AssertionError: Err-20 is greater than 18 expected [false] but found [true]
```

# Assertions by using Assert Class (4)

## Assertion Method

- **assertEquals():** Compares expected value with actual value. The assertion passes if values are the same.

```
@Test
public void testMethod1() {
    int actual = 10;
    int Expected = 10;
    Assert.assertEquals(actual, Expected, "Err- Both values are differ");
}

@Test
public void testMethod2() {
    String actual = "hello";
    String Expected = "Hello";
    Assert.assertEquals(actual, Expected, "Err- Both Strings are differ");
}
```

```
[RemoteTestNG] detected TestNG version 6.14.2
PASSED: testMethod1
FAILED: testMethod2
java.lang.AssertionError: Err-Both strings are differ expected [hello] but found [Hello]
```



# Assertions by using Assert Class (5)

## Assertion Method

- **assertNotEquals():** Compares expected value with actual value. The assertion passes if values are different

```
@Test
public void testMethod1() {
    int actual = 10;
    int Expected = 10;
    Assert.assertNotEquals(actual, Expected, "Err- Both values are same");
}

@Test
public void testMethod2() {
    String actual = "hello";
    String Expected = "Hello";
    Assert.assertNotEquals(actual, Expected, "Err- Both Strings are same");
}
```

```
[RemoteTestNG] detected TestNG version 6.14.2
PASSED: testMethod2
FAILED: testMethod1
java.lang.AssertionError: Err-Both values are differ
```

# Assertions by using Assert Class (6)

## Assertion Method

- **assertNull():** Compares the object with the null value. The assertion passes if the object is null

```
@Test
public void testMethod1() {
    Object obj1 = null;
    Assert.assertNull(obj1, "Err-Obj1 is not null");
}

@Test
public void testMethod2() {
    String obj2 = "Hello";
    Assert.assertNull(obj2, "Err-Obj2 is not null");
}
```

```
[RemoteTestNG] detected TestNG version 6.14.2
PASSED: testMethod1
FAILED: testMethod2
java.lang.AssertionError: Err-Obj2 is not null expected [null] but found [Hello]
```

# Assertions by using Assert Class (7)

## Assertion Method

- **assertNotNull():** Compares the object with the null value. The assertion passes if the object is not null

```
@Test
public void testMethod1() {
    Object obj1 = null;
    Assert.assertNotNull(obj1, "Err-Obj1 is null");
}

@Test
public void testMethod2() {
    String obj2 = "Hello";
    Assert.assertNotNull(obj2, "Err-Obj2 is null");
}
```

```
[RemoteTestNG] detected TestNG version 6.14.2
PASSED: testMethod2
FAILED: testMethod1
java.lang.AssertionError: Err-Obj1 is not null expected object to not be null
```

# Assertions by using Assert Class (8)

## Assertion Method

- **assertSame():** Compares the address of 2 objects. The assertion passes if the address of objects is the same

```
@Test
public void testMethod1() {
    Object obj1 = new Object();
    Object obj2 = obj1;
    Assert.assertSame(obj1, obj2);
}
```

```
@Test
public void testMethod2() {
    Object obj1 = new Object();
    Object obj2 = new Object();
    Assert.assertSame(obj1, obj2);
}
```

[RemoteTestNG] detected TestNG version 6.14.2

PASSED: testMethod1

FAILED: testMethod2

java.lang.AssertionError: expected [java.lang.Object@2286778] but found [java.lang.Object@4e9ba398]

- **assertNotSame:** Compares the address of 2 objects. The assertion passes if object addresses are different
- The behavior is opposite to assertsame.

# Assertions by using Assert Class (9)

## Assertion Method

- **Fail():** Allows the user to intentionally fail a test and specify a failure message

```
@Test
public void testMethod1() {
    Assert.fail("Intentionally Failing the Test");
}
```

```
[RemoteTestNG] detected TestNG version 6.14.2
FAILED: testMethod1
java.lang.AssertionError: Intentionally Failing the Test
```

# Assertions by using Assert Class (10)

## Behavior of Hard Assertions

```
@Test
public void testMethod1() {
    System.out.println("Checking 1st condition");
    Assert.assertEquals(10, 15);

    System.out.println("Checking 2nd condition");
    Assert.assertEquals("Hello", "Welcome");
}
```

```
[RemoteTestNG] detected TestNG version 6.14.2
Checking 1st condition
FAILED: testMethod1
java.lang.AssertionError: expected [15] but found [10]
```

- The first assertion validates and fails
- This results in `AssertionError`, terminating the execution of the test
- The second assertion does not execute

- If user wants to execute both the assertions in the test;
  - Hard Assertions will not support the scenario

# Assertions by using Assert Class (11)

## Use of Soft Assertions

- SoftAssert continues the test execution, even when few assertions have failed
- SoftAssert inherits all the methods present in the Assert Class
- It has `assertAll()` method, which collates all the failures and decides the test status: pass/fail

```
@Test
public void testMethod1() {

    SoftAssert saObj = new SoftAssert();
    System.out.println("Checking 1st condition");
    saObj.assertEquals(10, 15);

    System.out.println("Checking 2nd condition");
    saObj.assertEquals("Hello", "Welcome");

    saObj.assertAll();
}
```

```
[RemoteTestNG] detected TestNG version 6.14.2
Checking 1st condition
Checking 2nd condition
FAILED: testMethod1
java.lang.AssertionError: The following asserts failed:
    expected [15] but found [10],
    expected [Welcome] but found [Hello]
```

# Test Suite in TestNG



# Test Suite in TestNG (1)

## Test Suite Overview

- Test Suite is a collection of test cases designed to test the functionality of an application
- In TestNG, the test suite is created as an XML file (testng.xml)
- The XML file allows flexible configuration of tests and controls the test execution
  - Create multiple test cases
  - Include/Exclude Test Classes from execution
  - Control execution flow of tests
  - Parallel execution of tests
  - Include/Exclude Test Methods from execution
  - and many more

# Test Suite in TestNG (2)

## Exploring testing.xml file with Specified Classes

- **<suite>** tag represents the TestSuite, with the name "TestSuite01"
- **<test>** tag represents the TestCase, with the name "TestCase001"
- **<classes>** tag is a collection of multiple classes
- **<class>** tag represents the TestClass, with name "com.tests.TestClass01"
- This testng.xml file contains:
  - Test Suite
  - Test Cases
  - Test Classes

```
<?xml version="1.0" encoding="UTF-8"?>
<suite name="TestSuite01">
  <test name="TestCase001">
    <classes>
      <class name="com.tests.TestClass01" />
    </classes>
  </test> <!-- Test -->

  <test name="TestCase002">
    <classes>
      <class name="com.tests.TestClass02" />
    </classes>
  </test> <!-- Test -->
</suite> <!-- Suite -->
```

# Test Suite in TestNG (3)

## The testing.xml file with a Specified Package

- **<packages>** tag is collection of multiple packages
- **<package>** tag represents the complete package and classes inside the package, with name "com.tests"

```
<?xml version="1.0" encoding="UTF-8"?>
<suite name="TestSuite01">
    <test name="TestCase001">
        <packages>
            <package name="com.tests.*" />
        </packages>
    </test> <!-- Test -->
</suite> <!-- Suite -->
```

# Test Suite in TestNG (4)

## The testing.xml file with Specified Methods

- **<methods>** tag is a collection of multiple methods
- **<include>** tag represents the test method to execute
- **<exclude>** tag represents the test method not to execute

```
testng.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
3 <suite name="TestSuite01">
4   <test name="TC001">
5     <classes>
6       <class name="com.test.TestClass01">
7         <methods>
8           <include name="registerMethod"></include>
9           <include name="loginMethod"></include>
10          <exclude name="logoutMethod"></exclude>
11        </methods>
12      </class>
13    </classes>
14  </test> <!-- Test -->
15 </suite> <!-- Suite -->
```

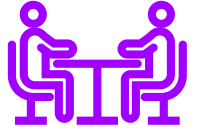
# Test Suite in TestNG (5)

## Use of Suite and Test Annotations with testng.xml

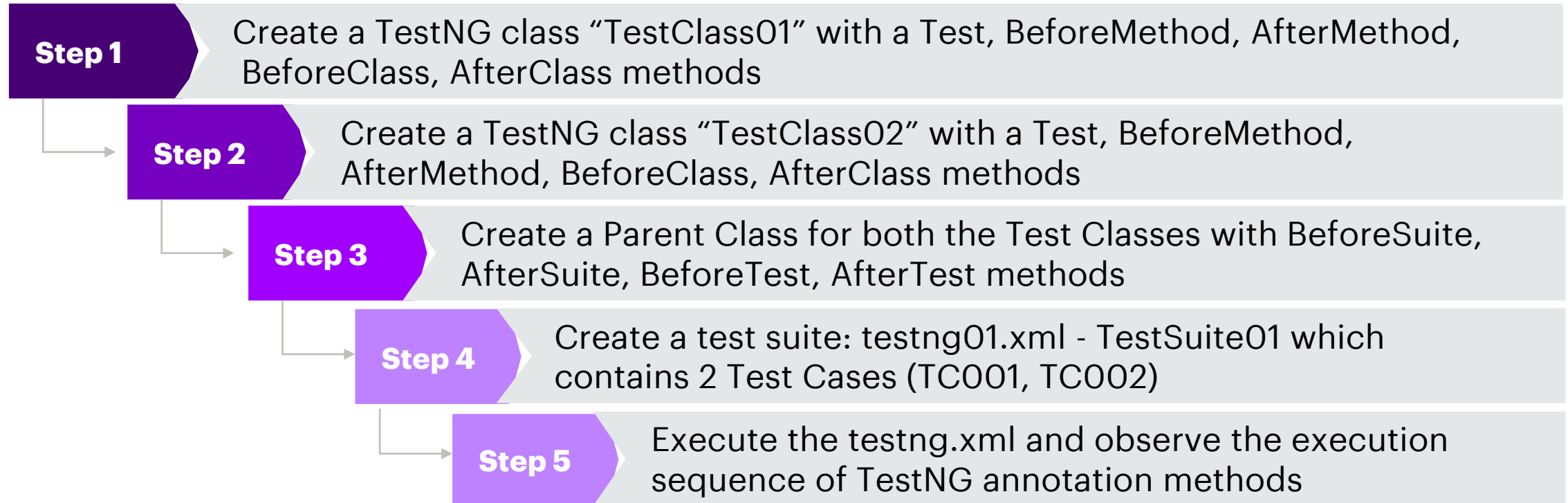
- BeforeSuite, AfterSuite, BeforeTest, and AfterTest annotations are applied across multiple test classes in the TestNG hierarchy (TestSuite → TestCase → TestClass → TestMethod)
  - BeforeSuite annotation method executes only once before all the tests in testng.xml
  - AfterSuite annotation method executes only once after all the tests in testng.xml
  - BeforeTest annotation method executes before any test cases <test> tag in testng.xml
  - AfterTest annotation method executes after any test cases <test> tag in testng.xml
- It is important to specify these annotations outside the test classes
- A parent/base class is created intentionally to write these annotations in the TestNG hierarchy
  - Parent Class → BeforeSuite, AfterSuite, BeforeTest, and AfterTest
  - Child Classes (Multiple Test Classes) → BeforeClass, AfterClass, BeforeMethod, AfterMethod, and Test

# Test Suite in TestNG (6)

## Demo: Use of Suite and Test Annotations with testng.xml



- Linda has learned the concept of Suite and Test annotations
- She decides to design a Java Project with multiple TestNG classes, annotations, and TestSuite
- Scenario



# Test Suite in TestNG (7)



## Demo Solution: Use of Suite and Test Annotations with testng.xml (1)

```
package com.testng;

import org.testng.annotations.AfterSuite;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeSuite;
import org.testng.annotations.BeforeTest;

public class BaseClass {

    @BeforeSuite
    public void beforeSuite() {
        System.out.println("In before suite method");
    }

    @AfterSuite
    public void afterSuite() {
        System.out.println("In after suite method");
    }

    @BeforeTest
    public void beforeTest() {
        System.out.println("In before test method");
    }

    @AfterTest
    public void afterTest() {
        System.out.println("In after test method");
    }
}
```

# Test Suite in TestNG (8)



## Demo Solution: Use of Suite and Test Annotations with testng.xml (2)

```
package com.testng;

import org.testng.annotations.AfterClass;

public class TestClass01 extends BaseClass {

    @BeforeClass
    public void beforeClass() {
        System.out.println("TestClass01: In before class method");
    }

    @AfterClass
    public void afterClass() {
        System.out.println("TestClass01: In after class method");
    }

    @BeforeMethod
    public void beforeMethod() {
        System.out.println("TestClass01: In before method");
    }

    @AfterMethod
    public void afterMethod() {
        System.out.println("TestClass01: In after method");
    }

    @Test
    public void testMethod1() {
        System.out.println("TestClass01: In test 1 method");
    }

    @Test
    public void testMethod2() {
        System.out.println("TestClass01: In test 2 method");
    }
}
```

```
package com.testng;

import org.testng.annotations.AfterClass;

public class TestClass02 extends BaseClass {

    @BeforeClass
    public void beforeClass() {
        System.out.println("TestClass02: In before class method");
    }

    @AfterClass
    public void afterClass() {
        System.out.println("TestClass02: In after class method");
    }

    @BeforeMethod
    public void beforeMethod() {
        System.out.println("TestClass02: In before method");
    }

    @AfterMethod
    public void afterMethod() {
        System.out.println("TestClass02: In after method");
    }

    @Test
    public void testMethod1() {
        System.out.println("TestClass02: In test 1 method");
    }

    @Test
    public void testMethod2() {
        System.out.println("TestClass02: In test 2 method");
    }
}
```



# Test Suite in TestNG (9)



Demo Solution: Use of Suite and Test Annotations with testng.xml (3)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Suite">
  <test thread-count="5" name="TC001">
    <classes>
      <class name="com.testng.TestClass01"/>
    </classes>
  </test> <!-- Test -->
  <test thread-count="5" name="TC002">
    <classes>
      <class name="com.testng.TestClass02"/>
    </classes>
  </test> <!-- Test -->
</suite> <!-- Suite -->
```

# Test Suite in TestNG (10)



Demo Solution: Use of Suite and Test Annotations with testng.xml (4)

```
[RemoteTestNG] detected TestNG version 7.3.0
In before suite method
In before test method
TestClass01: In before class method
TestClass01: In before method
TestClass01: In test 1 method
TestClass01: In after method
TestClass01: In before method
TestClass01: In test 2 method
TestClass01: In after method
TestClass01: In after class method
In after test method
In before test method
TestClass02: In before class method
TestClass02: In before method
TestClass02: In test 1 method
TestClass02: In after method
TestClass02: In before method
TestClass02: In test 2 method
TestClass02: In after method
TestClass02: In after class method
In after test method
In after suite method

=====
Suite
Total tests run: 4, Passes: 4, Failures: 0, Skips: 0
=====
```

# TestNG Report Generation

# TestNG Report Generation (1)

## TestNG Report

- TestNG Reports are the default HTML reports which are generated once the test cases are executed using TestNG.
- These reports help you to identify the information about test cases and the status of a project.

# TestNG Report Generation (2)

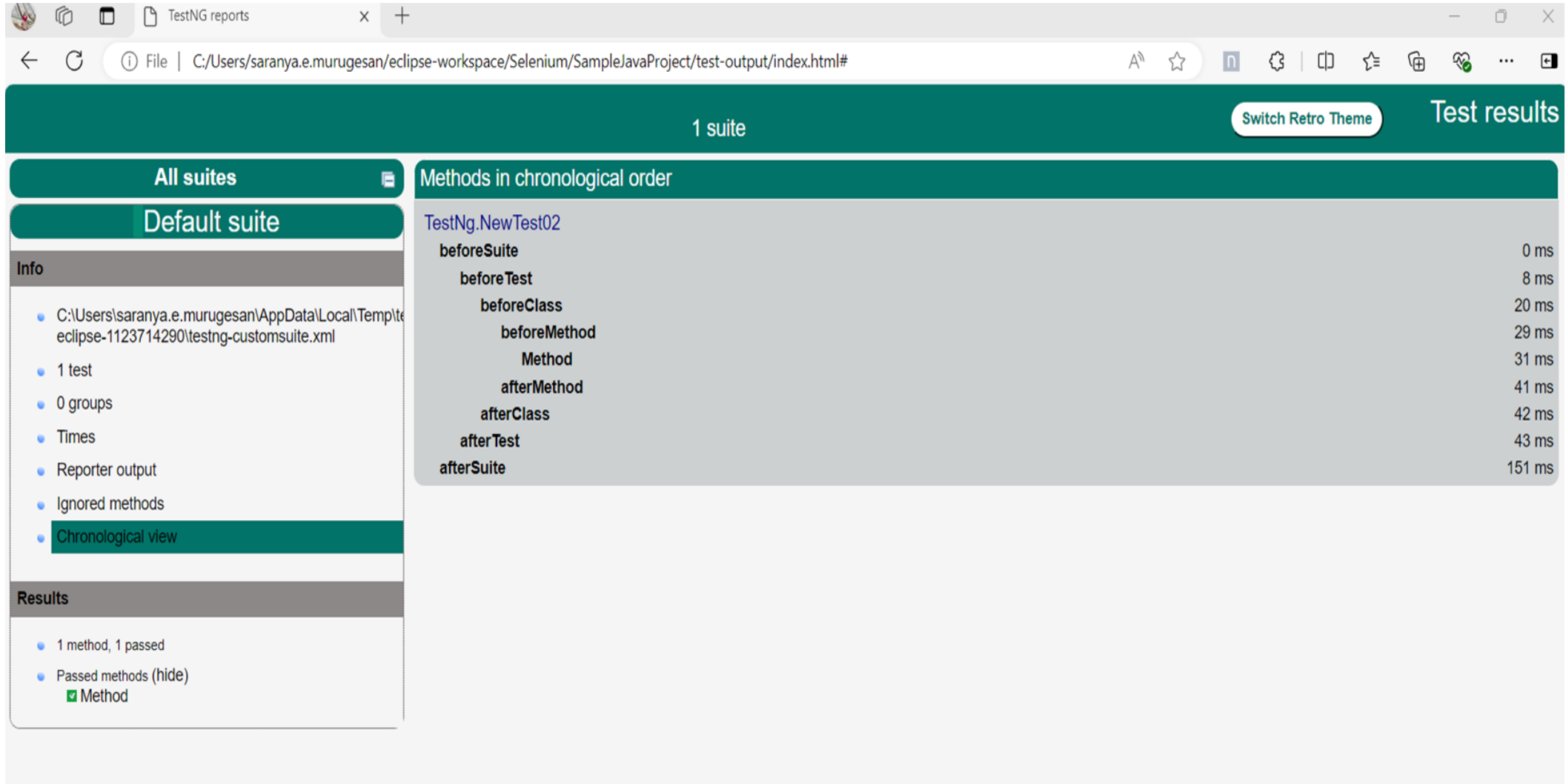
## TestNG Report

Steps to generate and view report

1. Execute TestNG.xml file
2. Refresh the project after the execution ,we will get test-output folder.
3. Expand the test-output folder
4. Click on option “index.html” for index report in eclipse
5. To open the report in web browsers, right click on the index.html and open with web browser

# TestNG Report Generation (3)

## Sample index.html report



TestNG reports

File | C:/Users/saranya.e.murugesan/eclipse-workspace/Selenium/SampleJavaProject/test-output/index.html#

1 suite [Switch Retro Theme](#) Test results

All suites

Default suite

Info

- C:\Users\saranya.e.murugesan\AppData\Local\Temp\teclipse-1123714290\testng-customsuite.xml
- 1 test
- 0 groups
- Times
- Reporter output
- Ignored methods
- Chronological view

Results

- 1 method, 1 passed
- Passed methods (hide)
  - Method

Methods in chronological order

TestNg.NewTest02

|              |        |
|--------------|--------|
| beforeSuite  | 0 ms   |
| beforeTest   | 8 ms   |
| beforeClass  | 20 ms  |
| beforeMethod | 29 ms  |
| Method       | 31 ms  |
| afterMethod  | 41 ms  |
| afterClass   | 42 ms  |
| afterTest    | 43 ms  |
| afterSuite   | 151 ms |

# TestNG Report Generation (4)

## TestNG Report

- Index reports, contains the index-like structure of different parts of the report.
- It will show methods executed, status of execution, failed tests, passed tests, etc.
- In Index Report, we have two parts.
  1. The left part contains the index, and this is the reason it is called an index report.
  2. The right part contains the explored content of that index.

# Knowledge Check

## Question 01



Which is the correct hierarchy in TestNG Test Suite?

a) Suite → Case → Class → Method

b) Suite → Class → Case → Method

c) Suite → Class → Method

d) Suite → Case → Method



# Knowledge Check

## Question 02



Which of these annotations executes only once before the test methods?

a) BeforeMethod

b) BeforeClass

# Knowledge Check

## Question 03



@BeforeTest executes only once, irrespective of multiple the test cases in test suite

a) True

b) False

# Module Summary

Now, you will be able to:

- Describe the TestNG Framework and its features
- Explore the installation of the TestNG plugin for Eclipse
- Elaborate the usage of TestNG test cases along with Annotations
- Realize the functioning of Assertions using Assert class
- Describe the functional offering of Test Suite



**Thank You**

